

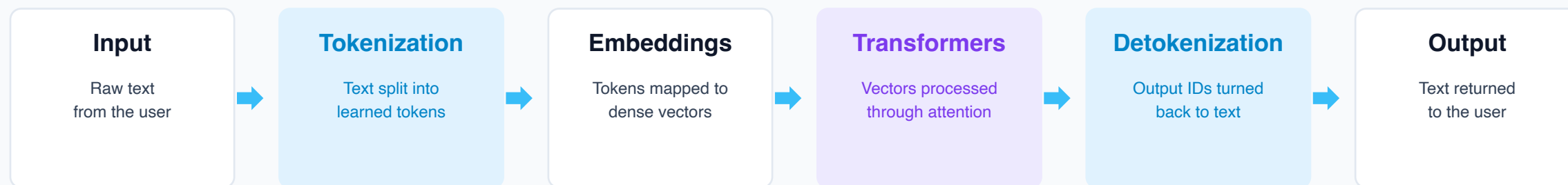
Tokenization

BPE • WordPiece • SentencePiece

How raw text becomes the discrete units a Transformer can actually process — and why that choice drives vocabulary size, sequence length, context windows and cost.

Where Tokenization Fits in an LLM

Before we go deep on tokenization, here is the full path a prompt takes through a language model.



TODAY'S FOCUS

This lecture is entirely about the second box — Tokenization — and its mirror image, Detokenization, at the end. Everything in between (embeddings, the Transformer) assumes tokenization has already happened.

WHY START HERE

Tokenization is the very first thing that happens to a prompt and the very last thing that happens to a response — get it wrong and nothing downstream can fix it.

Learning Objectives

1. Why tokenization exists

Explain why NLP models and LLMs cannot consume raw text directly.

2. Three families

Differentiate character-, word- and subword-level tokenization.

3. Limits of word-level

Vocabulary explosion, OOV words, and lost morphology.

4. Byte Pair Encoding

Work through BPE merge-by-merge on a small corpus.

5. WordPiece

Understand its objective and how it differs from BPE.

6. SentencePiece

See why raw-text tokenization matters for multilingual LLMs.

7. Cost & capacity

Relate vocabulary size, sequence length and context windows.

8. The full pipeline

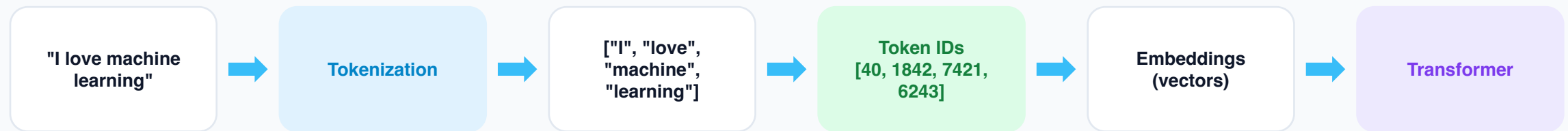
Text → Tokens → Token IDs → Embeddings → Transformer.

By the end of the session you should be able to look at any tokenizer output and explain what it did and why.

What Is Tokenization?

“If I give an LLM the sentence ‘I love machine learning’, what exactly does the model see?”

It does not see text. The pipeline is:



KEY STATEMENT

Tokenization converts raw text into discrete units called tokens that a neural network can process.

CAVEAT

The exact token IDs depend entirely on the tokenizer and model. Two models will not agree on them.

Everything downstream — attention cost, context window, API billing, RAG chunk sizing — is denominated in the units this one step produces.

Why Not Feed Raw Text to a Neural Network?

Neural networks operate on numbers — not characters.

TEXT BECOMES INTEGERS

```
I love NLP
  |
  v
[45, 892, 7312]
```

These integers are called token IDs. Each ID is then mapped to a dense vector.

ID BECOMES A VECTOR

```
"cat"
-> token ID = 5821
-> embedding
-> [0.21, -0.73, 0.45, ..., 0.18]
```

The Transformer never sees the string “cat” — only the vector that ID 5821 points to.



CONSEQUENCE

The tokenizer defines the model’s entire input alphabet. It is chosen before training and frozen for the life of the model — you cannot swap it afterwards without retraining the embedding matrix.

What Exactly Is a Token?

A token does not necessarily mean a word.

STRAIGHTFORWARD

I love NLP

➡ `["I", "love", "NLP"]`

NOT SO STRAIGHTFORWARD

unhappiness

➡ `["un", "happiness"]`

... or any of these, depending on the tokenizer's vocabulary:

`["un", "happiness"]`

`["un", "happy", "ness"]`

`["u", "n", "h", "a", "p", "p", "..."]`

DEFINITION

A token is a unit chosen by a tokenizer according to a particular tokenization algorithm and a particular learned vocabulary. It is an artefact of the algorithm — not a linguistic fact.

This is why token counts differ across models for exactly the same sentence.

The Tokenization Problem

English has millions of word forms — and new ones appear constantly: unbelievableness, electrification, counterrevolutionary.

PROBLEM 1

Huge vocabulary

One entry per word means a 1,000,000-row embedding matrix:

$1,000,000 \times \text{embedding_dim}$

Expensive in memory, and the output softmax gets just as large.

PROBLEM 2

Out-of-vocabulary words

Training saw play, played, playing.
A user types playfulness.

The word was never seen as a whole unit → the model emits [UNK] and the information is simply lost.

PROBLEM 3

Morphology is thrown away

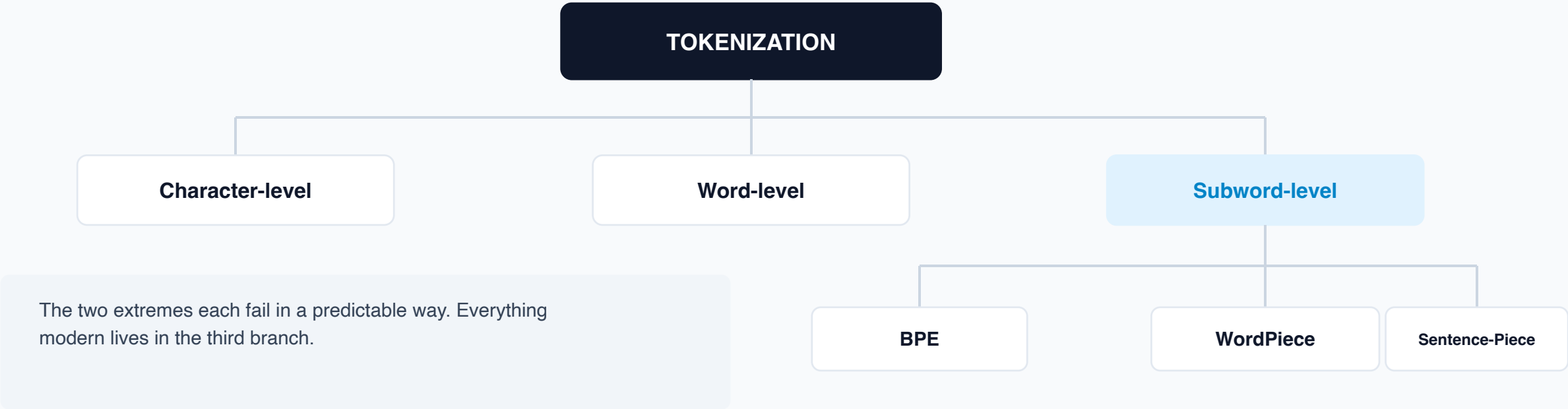
play, played, playing, player, playful, playfulness clearly share structure.

A word-level tokenizer treats all six as unrelated, arbitrary integers.

WHAT WE ACTUALLY WANT

A vocabulary small enough to be cheap, complete enough to never fail on an unseen word, and structured enough that related words share pieces.

Three Major Approaches



Approach	Vocabulary	Sequence length	OOV
Character	Tiny (~100)	Very long	Almost none
Word	Enormous	Short	Severe
Subword	Moderate (30k–100k)	Moderate	Almost none

Character-Level Tokenization

Split into individual characters.

```
hello      ->  h e l l o
transformer -> t r a n s f o r m e r      (11 tokens)
```

ADVANTAGES

- Very small vocabulary
- Almost no OOV problem at all
- Can represent arbitrary, never-seen words

DISADVANTAGES

- Sequences become very long
- More computation and memory
- Attention is quadratic — length hurts twice

THE REAL COST

“transformer” costs 11 tokens instead of perhaps 1–3. Since self-attention scales with the square of sequence length, a 4× longer sequence is roughly 16× the attention cost — for the same sentence.

Word-Level Tokenization

```
I love machine learning -> ["I", "love", "machine", "learning"]
```

ADVANTAGES

- Intuitive and easy to explain
- Short sequences
- Tokens map to meaningful linguistic units

PROBLEMS

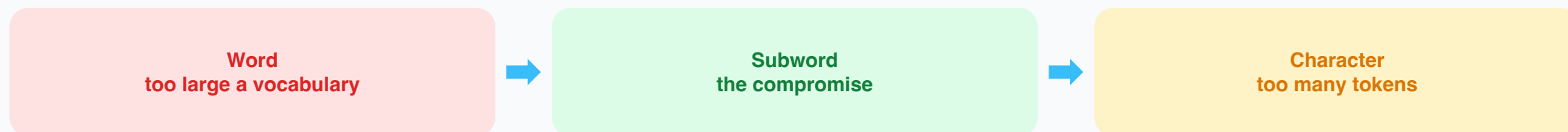
- Vocabulary becomes huge
- Unseen words collapse to a single [UNK]
- Related word forms share nothing

```
unhappiness -> not in vocabulary -> [UNK]
```

```
The model has no idea whether that was a person, a place, or a feeling.
```

One [UNK] can silently destroy the meaning of an entire sentence — and you will not see an error.

Subword Tokenization: The Compromise



Instead of committing to whole words or bare characters, learn useful pieces:

```
unhappiness  ->  un  +  happy  +  ness
```

Now the model can recognise pieces of a word it has never seen as a whole.

This is the central idea behind all three algorithms in this lecture:

BPE

WordPiece

SentencePiece

They differ in how the pieces are learned — not in why.

Why Subwords Work So Well

A word tokenizer sees five unrelated entries:

```
teach
teacher
teaching
teachable
teachers
```

5 separate vocabulary entries

A subword tokenizer reuses shared pieces:

```
teach
teach + er
teach + ing
teach + able
teach + er + s
```

vocab: teach, er, ing, able, s

The stem is learned once and shared by every derived form:

teach

er

ing

able

s

THE PAYOFF

Smaller vocabulary and better handling of rare words at the same time — plus the embeddings for shared pieces get far more training signal than any single rare word form would.

BPE — Byte Pair Encoding

THE ENTIRE ALGORITHM IN ONE LINE

Repeatedly merge the most frequent pair of adjacent symbols.

ORIGIN

BPE began life as a data compression algorithm. It was later adapted for NLP tokenization — same mechanic, different purpose.

WHY IT CAUGHT ON

Purely frequency-driven, trivially reproducible, and it gives you a knob — the number of merges — that directly sets vocabulary size.

We will now run it by hand on a three-word corpus.



Stop when the vocabulary reaches the size you asked for. That stopping point is the only real hyperparameter.

BPE Worked Example — Step 1

Training corpus: low, lower, lowest

1. REPRESENT EVERY WORD AS CHARACTERS

```
l o w
l o w e r
l o w e s t
```

2. COUNT ADJACENT PAIRS

```
l o      o w      w e
e r      e s      s t
```

The pair `l o` is the most frequent — it appears in all three words.

MERGE: `l` `+` `o` `→` `lo`

3. THE CORPUS AFTER THE FIRST MERGE

```
lo w
lo w e r
lo w e s t
```

BPE Worked Example — Continuing

Recount the pairs on the updated corpus. Now lo + w is the most frequent.

MERGE 2

```
MERGE:  lo + w -> low
```

```
low  
low e r  
low e s t
```

MERGES 3 AND 4

```
MERGE:  e + r -> er  
MERGE:  e + s -> es
```

```
low  
low er  
low es t
```

Continue until the target vocabulary size is reached. Eventually:

```
low  
lower  
lowest
```



```
low  
low + er  
low + est
```

WATCH FOR THIS

The exact merges depend on corpus frequencies and where you stop. Change the corpus and you get a different vocabulary — which is precisely why two models tokenize the same sentence differently.

The BPE Algorithm

1

Start with characters or bytes

Every word becomes a sequence of atomic symbols.

2

Count adjacent token pairs

Across the whole training corpus.

3

Find the most frequent pair

Ties broken by implementation convention.

4

Merge that pair

Replace every occurrence in the corpus.

5

Add the merged token to the vocabulary

And record the merge rule.

6

Repeat until the target vocabulary size

WHAT TRAINING PRODUCES

Merge rules,
in learned order:

```
l  + o  -> lo
lo + w  -> low
e  + r  -> er
e  + s  -> es
es + t  -> est
...
```

The order matters — merges
are applied in this sequence.

IMPORTANT

BPE training learns merge rules. Tokenization then simply replays those rules on new text.

BPE Training vs BPE Tokenization

TRAINING

Learn merge rules from a corpus

Runs once, offline, on a large corpus.

Output: an ordered list of merges + a vocabulary.

INFERENCE

Apply those rules to new text

Runs on every prompt, every time.

No counting, no learning — pure replay.

REPLAYING THE LEARNED MERGES ON A NEW WORD

Input: lowest

l o w e s t

-> l o w e s t	(apply l + o -> lo)
-> l o w e s t	(apply lo + w -> low)
-> l o w e s t	(apply e + s -> es)
-> l o w e s t	(apply es + t -> est)

Final: [low, est]

Deterministic: the same text and the same merge list always give the same tokens.

Why BPE Beats Word Tokenization

Suppose the model has never seen the word:

unbelievability

WORD TOKENIZER

[UNK]

All internal structure is discarded.

BPE

un + believe + ability

Three pieces the model has seen thousands of times.

THE KEY PROPERTY

Even when the complete word is unseen, the model can still process meaningful pieces — and the negation prefix un- carries exactly the meaning it carries everywhere else.

This is what makes subword models robust to typos, neologisms, product names, and code identifiers — all of which a fixed word vocabulary would simply throw away.

Byte-Level BPE

Modern systems often run BPE over bytes rather than Unicode characters. Starting from the 256 possible byte values gives one very strong guarantee:

Almost any text can be represented — no unknown-token failure caused simply by an unseen character.

Particularly valuable for:

Multilingual text

Emojis

Rare symbols

URLs

Source code

Misspellings

hello 🤔

The emoji is never in the vocabulary as a character — but its byte sequence always is.

TRADE-OFF

Nothing is free: an emoji or a CJK character may cost several byte tokens rather than one.

WordPiece

Popularised by BERT. It looks like BPE — both build subword vocabularies — but the learning objective differs.

BPE

Merge the most frequent pair.

A purely frequency-driven, greedy criterion.

WordPiece

Select the subword units that give the best likelihood / objective on the training data.

Frequency matters, but so does usefulness.

“WordPiece is not exactly BPE.” they are closely related subword algorithms, but their vocabulary-learning criteria differ. BPE asks “what is most frequent?”; WordPiece asks “what most improves the objective?”

A typical WordPiece vocabulary fragment:

play

##ing

##ed

##er

un

happy

##ness

What Does ## Mean?

The ## prefix marks a piece intended to continue an existing word.

```
playing    ->  play    ##ing
happiness  ->  happy   ##ness
```

WHAT IT IS

play + continuation(“ing”)

The tokenizer distinguishes a word-initial piece from a mid-word piece.

WHAT IT IS NOT

play ing

These are not two independent words.
##ing on its own is not a word at all.

WHY THE DISTINCTION MATTERS

Without it, the tokenizer could not tell whether a piece began a word or continued one — and detokenization (putting spaces back) would be ambiguous.

WordPiece Tokenization — Greedy Longest Match

Vocabulary contains:

un

happy

##ness

##ful

At each position the tokenizer takes the longest piece that is in the vocabulary, then continues from what remains.

GREEDY WALKTHROUGH

Input: unhappiness

```
longest match -> un
remaining     -> happiness
```

```
longest match -> happy
remaining     -> ness
```

```
longest match -> ##ness
remaining     -> (none)
```

Final: un + happy + ##ness

SIMPLIFIED VIEW

This is the textbook picture. Real implementations add details — word-boundary handling, a fallback to characters, and an [UNK] of last resort when no piece matches at all.

BPE vs WordPiece

Feature	BPE	WordPiece
Basic idea	Pair merging	Likelihood / objective-based vocabulary learning
Popular with	GPT-style tokenizers, many others	BERT
Training	Frequency-based merges	Select useful subwords based on an objective
Tokenization	Apply learned merges	Often greedy longest-match
Continuation marker	Depends on tokenizer	## commonly used
OOV handling	Excellent with a suitable vocabulary	Excellent with a suitable vocabulary

IMPORTANT CLARIFICATION

There are many implementations and variants. We should not assume every modern tokenizer follows exactly the textbook version — check the actual tokenizer config of the model you are using.

SentencePiece — and the Whitespace Problem

Most tokenizers assume you can pre-split on spaces. That assumption is English-shaped.

ENGLISH — SPACES DO THE WORK

I love machine learning

Spaces naturally separate words, so
word1 space word2 works fine.

CHINESE / JAPANESE / THAI

我喜欢机器学习

No spaces separate the words at all.
Pre-tokenization has nothing to split on.

THE DESIGN DECISION

SentencePiece treats the input as a raw character stream — it does not require whitespace-based pre-tokenization. That is the major difference from BPE and WordPiece as usually deployed.

A tokenizer that depends on `word1 space word2 space word3` cannot be a general multilingual tokenizer.
SentencePiece removes the dependency instead of special-casing each language.

SentencePiece: Spaces Become Tokens

Rather than discarding whitespace, SentencePiece encodes it as part of the token itself.

```
I love NLP
  |
  v
_I _love _NLP
```

THE MARKER

 represents a word boundary
(the space that preceded the piece).

WHY THIS IS ELEGANT

Detokenization becomes exact and lossless: replace `_` with a space and concatenate. No heuristics about where spaces “probably” belonged.

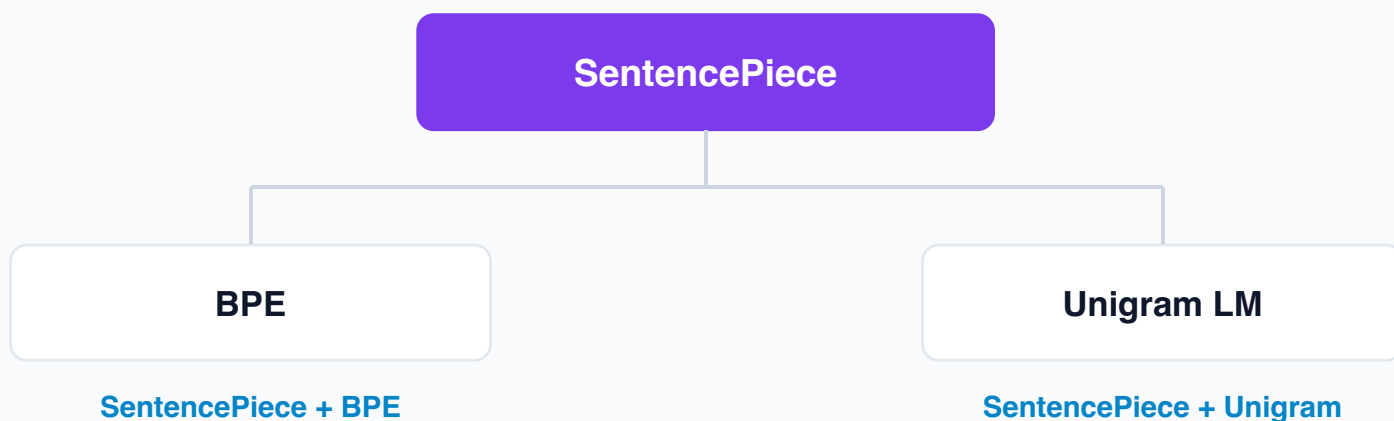
CONSEQUENCE

"NLP" and " NLP" are different tokens. This is why leading spaces matter in prompts, and why token counts change when you reformat text.

SentencePiece Is a Framework, Not an Algorithm

FREQUENTLY MISUNDERSTOOD

SentencePiece is not synonymous with BPE. It is a tokenization framework / library that can train different subword models.

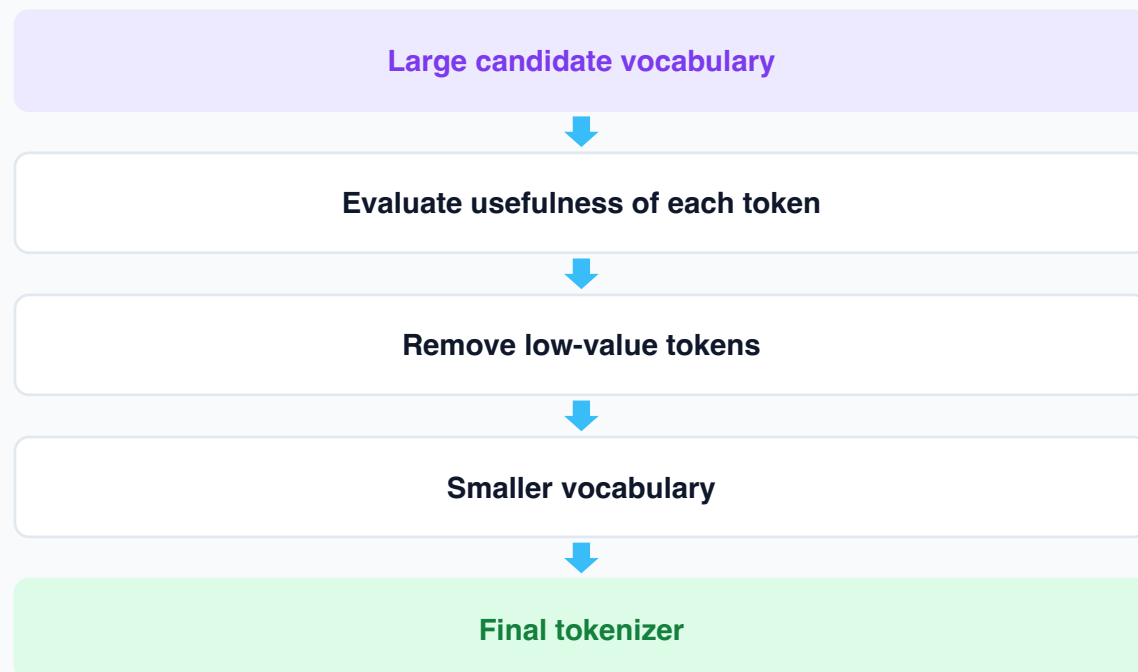


SO WHEN SOMEONE SAYS “THIS MODEL USES SENTENCEPIECE”

... they have told you how the text is pre-processed (raw stream, __ boundaries) but not which vocabulary-learning algorithm was used. Always ask the follow-up: BPE or Unigram?

The SentencePiece Unigram Model

Unigram works in the opposite direction from BPE. Instead of building up from characters by merging, it starts large and prunes down.



BPE — bottom-up

Start with characters, merge the most frequent pair, repeat.

Unigram — top-down

Start with many candidates, prune the least useful under an LM objective.

THE PRACTICAL DIFFERENCE

Unigram assigns probabilities to whole segmentations, so it can score alternatives rather than committing to one deterministic merge path — and it can sample segmentations during training as regularization.

Both end at the same place: a fixed vocabulary of subword pieces. They just search for it from opposite ends.

Example: Competing Segmentations

For the word **unhappiness** several segmentations are legal:

un + happiness

2 tokens — coarse

un + happy + ness

3 tokens — morphologically clean

u + n + happiness

3 tokens — wasteful

HOW UNIGRAM CHOOSES

The Unigram model assigns a probability to each possible segmentation and selects the likely one(s). This is more flexible than deterministic pair merging, where the merge order alone decides the outcome.

BPE BY CONTRAST

BPE has no notion of a “better” segmentation. It replays its merge list in order, and whatever falls out is the answer. That makes it fast and reproducible, but it cannot reconsider a bad early merge.

SentencePiece vs BPE — Settling the Confusion

This is a frequent exam question, and a frequent interview question.

BPE

An algorithm.

Starts from smaller units and MERGES frequent pairs to build a vocabulary.

SentencePiece

A framework.

Processes raw text directly and can use either BPE or Unigram underneath.

BPE $\neq$ SentencePiece

SentencePiece CAN implement BPE

ONE-SENTENCE ANSWER

They are not alternatives at the same level: BPE is a vocabulary-learning algorithm; SentencePiece is a raw-text tokenization framework that can run BPE or Unigram as its learner.

BPE vs WordPiece vs SentencePiece

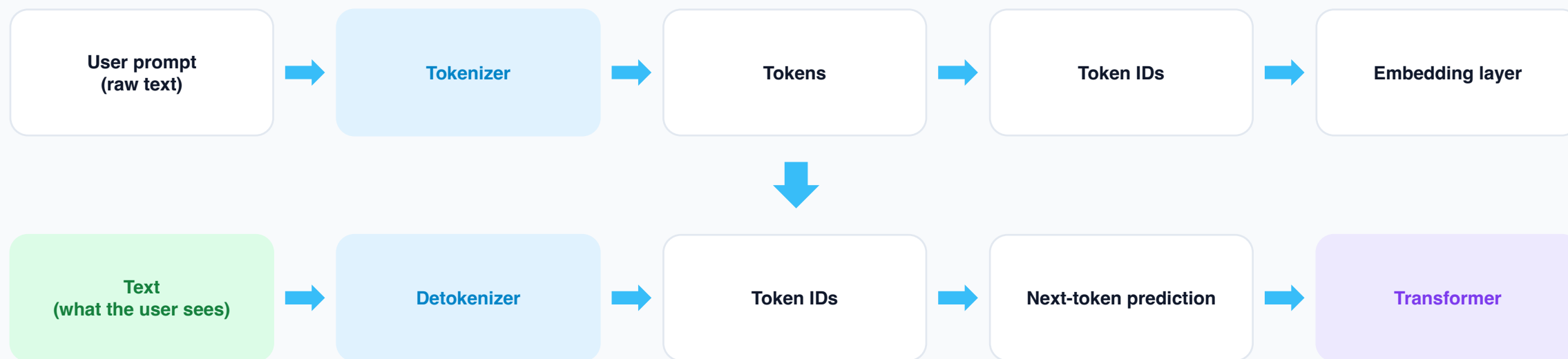
Property	BPE	WordPiece	SentencePiece
Type	Subword algorithm	Subword algorithm	Tokenization framework
Basic strategy	Frequent pair merging	Likelihood / objective	BPE or Unigram
Raw whitespace required?	Depends on implementation	Usually pre-tokenized	No
Word boundary marker	Implementation-dependent	## continuation	__ prefix
OOV	Very low	Very low	Very low
Multilingual	Good	Good	Excellent
Famous association	GPT-style models	BERT	T5 / many multilingual models

IF YOU REMEMBER ONE ROW

Remember the first: two of these are algorithms and one is a framework. Almost every confusion about tokenizers traces back to that row.

Tokenization in Modern LLMs

The complete round trip, from what the user types to what they read back:



(the lower row runs right to left — the model predicts IDs, and the detokenizer turns them back into text)

WHY THIS MATTERS IN PRACTICE

The model generates one token ID at a time. Streaming output looks character-by-character to a user, but the unit of generation is a token — which is why partial words sometimes appear, and why stop sequences must be expressed in tokens, not characters.

A Worked Example, End to End

Input: "Artificial intelligence is amazing."

Tokens:

```
["Artificial", " intelligence", " is", " amazing", "."]
```

Token IDs:

```
[15423, 9551, 318, 4992, 13]
```

Embeddings:

```
15423 -> [0.14, -0.62, ...]  
9551   -> [0.88,  0.03, ...]    -> Transformer  
318    -> [-0.21, 0.47, ...]
```

NOTE THE SPACES

"intelligence" includes its leading space. The space is part of the token, not a separator between tokens.

NOTE THE IDS

These IDs are illustrative only. Different models have completely different vocabularies and completely different IDs.

Punctuation is its own token here — five tokens for four words.

Why Tokenization Drives LLM Cost

LLMs charge and compute per token — not per word.

A ROUGH ENGLISH RULE OF THUMB

100 words → **≈ 130 tokens**

BUT THE RULE BREAKS DOWN

Source code, non-Latin scripts, long identifiers and unusual formatting can produce dramatically different token counts.

Input length



Number of tokens



Memory + computation + cost

Tokenization therefore directly affects:

Context-window usage

API cost

Inference speed

Memory requirements

If you are optimising spend, the tokenizer is the first thing to measure — not the last.

Context Window ≠ Word Count

Context window = 128K tokens

This does not mean 128,000 words. It means approximately 128,000 tokenizer tokens.

How much text actually fits depends entirely on the tokenizer:

Content type	Roughly	Fits in 128K tokens
Plain English prose	~1.3 tokens per word	~95,000 words
Source code	Often far more per line	Substantially less
Non-Latin scripts	Can be several tokens per character	Much less

PRACTICAL IMPLICATION

The tokenizer — not the model architecture — determines how much of a document you can actually fit. Two models advertising the same context window may hold very different amounts of your text.

Why Tokenization Matters for RAG

Indexing pipeline:



Retrieval pipeline:



CHUNKS TOO LARGE

Too many tokens — the context window fills with mostly irrelevant text, and cost rises with it.

CHUNKS TOO SMALL

Context is severed mid-thought — the retrieved fragment no longer carries its meaning.

THE LINK BACK TO LECTURE 4

Chunk size in RAG is specified in tokens, measured by the tokenizer. Tokenization is therefore a direct input to your chunking strategy — not an implementation detail beneath it.

Tokens Are Not Words

"hello"



1 token

"unbelievable"



un + believe + able

"Hello! "



Hello + !

Punctuation is a token. Whitespace influences tokenization. Case can change the split.

Never assume 1 word = 1 token.

The same sentence in four languages can cost very different numbers of tokens:

ENGLISH

I love AI

HINDI

मुझे AI पसंद है

CHINESE

我喜欢人工智能

JAPANESE

人工知能が好きです

Different token efficiency → different context consumption → different inference cost for the same meaning.

The Core Trade-Off

Vocabulary size



Sequence length

You cannot reduce both. Every tokenizer design is a point on this curve.

Large vocabulary

- Fewer tokens per sentence
- Large embedding and output matrix
- More parameters spent on rare pieces

Small vocabulary

- More tokens per sentence
- Smaller embedding matrix
- Longer sequences — quadratic attention cost

WHERE SUBWORDS SIT

Subword tokenization exists to find a practical balance — typically 30,000–100,000 pieces, which keeps both the embedding matrix and the sequence length manageable.

Character tokenization sits at one extreme; word tokenization at the other. Neither is used in practice.

Remember These Seven Things

1

Token $\neq$ Word

A token can be a word, subword, character, punctuation mark, space-related unit or byte sequence.

2

Word tokenization

Simple, but suffers from a large vocabulary and the OOV problem.

3

Character tokenization

Solves OOV, but creates very long sequences.

4

Subword tokenization

The practical compromise — and what every modern LLM uses.

5

BPE

Repeatedly merge the most frequent pair.

6

WordPiece

Learn useful subwords based on a likelihood / objective.

7

SentencePiece

A raw-text tokenization framework that can use BPE or Unigram.